Continental
The Future in Motion

Automotive Linux Security

Florin Anton

www.continental.com/ro-ro

Agenda

1. **Introduction**

2. Make Sure You Run the Right Code

- 2.1 Before Booting
- 2.2 When Booting
- 2.3 After Booting

3.Keeping Secrets Secret

- 3.1 Isolation
- 3.2 Virtualization
- 3.3 Access Controls
- 3.4 Restricting Permissions
- 3.5 Leverage Hardware Capabilities

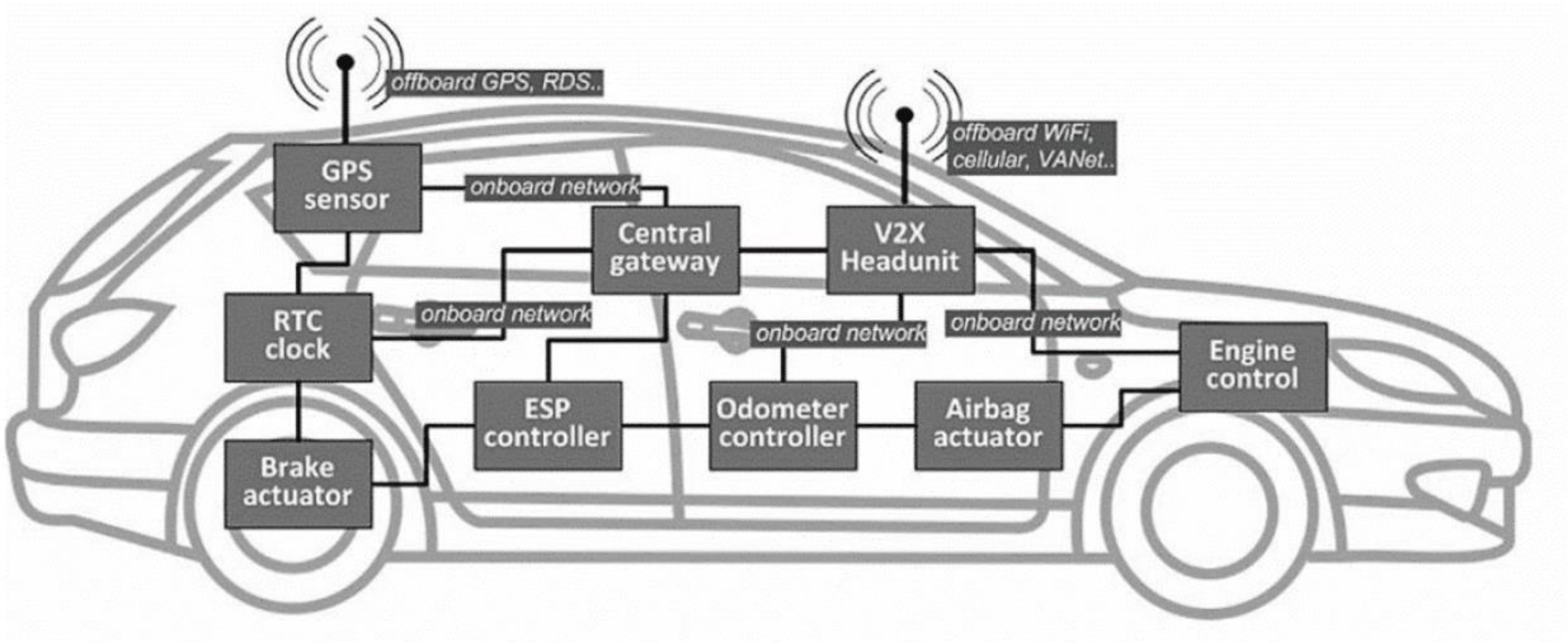
4.Connectivity with the Outside World

- 4.1 Network Connections
- 4.2 Removable Storages
- 4.3 Connectivity to the Cloud

5.Upgrading Software

- 5.1 Upgrading Baseline System
- 5.2 Upgrading Applications
- 5.3 Very Long Time Support

Introduction



2. Make Sure You Run the Right Code

› **Before Booting**

- Automotive systems should be protected even against people who have full physical access to the device:
  - Protecting the hardware: In production interfaces like JTAG or others, low level mechanisms to introduce code in RAM need to be disabled through an irreversible fuse feature. Allowing JTAG is allowing anyone to do anything from inside the processor. Securing the system is simply not possible if development/debug capabilities remain open.
  - Making sure the bootloader is the expected one: In production, a safe option is to permanently fuse the bootloader code in a ROM directly on the SoC and forbid updates. Alternative boot methods should be forbidden too, as soon as the boards quit factory or development labs.

2. Make Sure You Run the Right Code

- After Booting
- Before accepting an application, the system should:
  - Check the code
    - Applications should be downloaded within a sealed package containing not only the application, but also its configuration, security rules, attached data. Application packages should embed cryptographic signatures to allow integrity checks, typically based on some hashes on content, with guaranteed origin and global consistency.
  - Handle dependencies
    - Installing or upgrading software usually requires checking that its dependencies (the other software components that are required for proper operation) are satisfied by the system
  - Install the application
    - Installation processes should activate security rules into the file system. Until the full installation process is terminated, the application should not be launched. Adding new applications should impose the update of a trusted repository with new corresponding file properties and hashes. Doing so requires using a secret key which can be specific to the hardware
  - Start the application
    - The system should check application integrity before starting. This check has a performance cost and should take into account initial user experience. To limit impacts on performance, this operation could be cached to speed up any further attempt to restart the application.

Internal

Keeping Secrets Secret

Isolation

- **Keeping secrets implies building big walls with a few well-guarded doors. This fortress policy is also called isolation and can be implemented using numerous techniques:**

Virtualization:

- hardware: access is achieved through a hypervisor [eg: KVM5]
- software: relies on Linux namespace & CGroup as Docker or OpenContainer6

Access control:

- **discretionary (regulated by owners)**
- **mandatory (regulated by security rules)**

Permissions restrictions:

- **by user**
- **by applications**

All these techniques are complementary and can be used together or independently.

Virtualization

The virtualization goal is to create within a unique host system the illusion of multiple private sub-systems or guest systems. Any virtualization system provides mechanisms to limit resources usage (CPU, Ram, Filesystem...).

HW virtualization

Each virtual machine runs a private operating system using hardware separation inside the processor under the supervision of minimal software called a “hypervisor”. Its potential use in the automotive could be to keep isolated the master ECU from the IVI while using a single hardware board.

SW virtualization

The soft virtualization, also called containerization, is the application of a Linux mechanism called “unsharing”. This virtualization still has a cost but remains very light compared to hardware virtualization. In automotive it could be used to isolated untrustworthy applications from baseline services.

OH SNAP…

- Snaps are a secure and scalable way to embed applications on Linux devices. A snap is an application containerised with all its dependencies. A snap can be installed using a single command on any device running Linux. With snaps, software updates are automatic and resilient. Applications run fully isolated in their own sandbox, thus minimising security risks.
- Snaps can be created by anyone - existing software can be packaged as a snap or new software can be built from scratch using snap packaging.

Reliable	Snaps use transactional updates, meaning that if for any reason an update you push to your snap fails, the snap will roll back to its last stable state
Modular	Snaps are reusable, they enable a loosely-coupled software architecture for embedded software and are compatible across architectures
Robust	With snaps, software updates are automatic and over-the-air (OTA), meaning your software is never out-of-date
Optimised	Snaps harness delta updates, minimising the storage and bandwidth needed when updating software.

Access Controls

Access Controls are used to filter accesses to read, write or execution mode of resources exposed by Linux file-systems. This include files and directories but also other resources like RAM, processes, devices and network interfaces.

There are two main classes of access control:

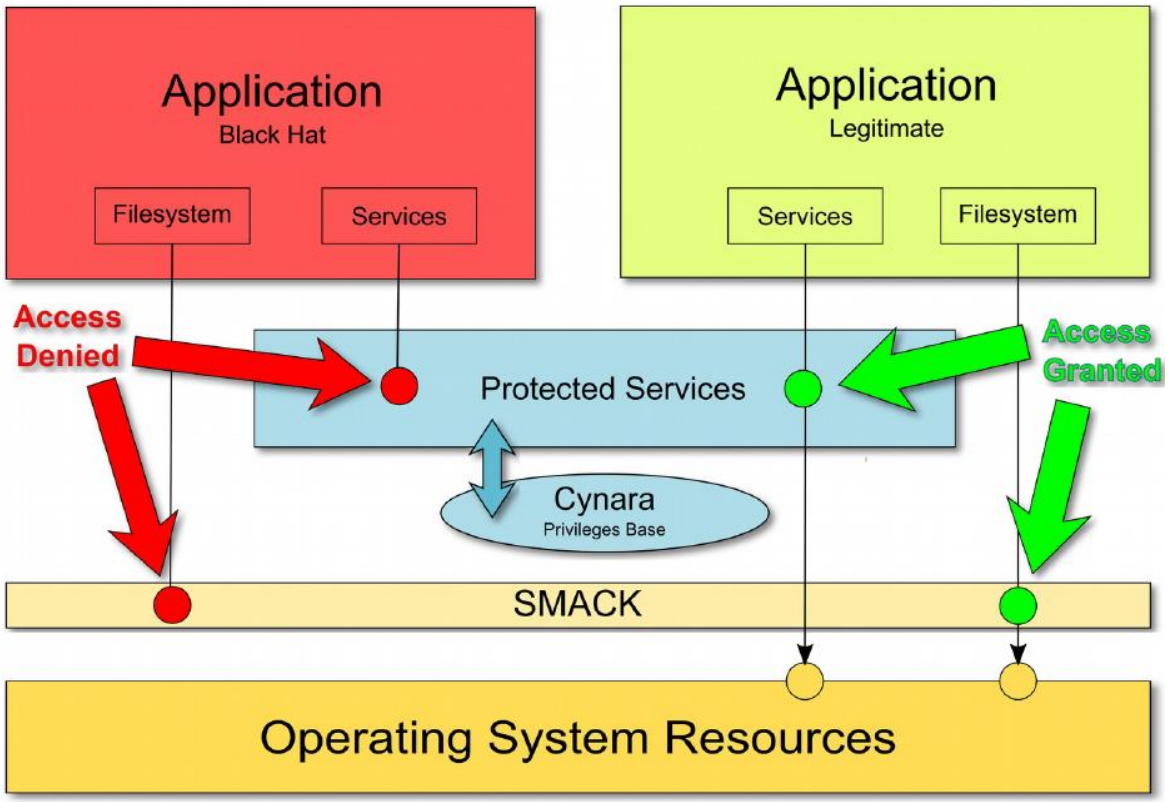
- **Discretionary Access Control (DAC)**: As the owner of a resource, you choose who is and isn't allowed to use your resource. This is the traditional Unix access control. They define and use the concepts of "user", "group" and "others". Users map to real persons or logical entities and are members of one or more groups.
- **Mandatory Access Control (MAC):** Who can use your resource is decided by a piece of software called "security manager". The mandatory access control philosophy is based on the idea that the owner of a resource may not be a security expert. Today standard Linux kernels supports three security modules (LSM) that implement three different Mandatory Access Control models:
    - **SELinux** (Security-Enhanced Linux) is the most complete solution but also quite complex. SELinux is used by RedHat, CentOS, and Android.
    - **AppArmor** (Application Armor) is a light solution with some limitations. It is used by Ubuntu and SUSE.
    - **Smack** (Simplified Mandatory Access Control Kernel) solution enforces simplicity and is mainly used in the embedded world.

Internal

Restricting Permissions

›Access control [MAC/DAC] can be used for global security and isolation of a full system:

- Restricting permissions by user: Multiple users can be associated to a given car, such as the owner, the mechanic, the driver, and the passengers. The privileges of these users are not equivalent and some should have more rights than others. This is managed by permissions associated with users or profiles.
- Restricting permissions by application: An application has to request the needed permissions for proper operation. When requested permissions are granted, the application can then access protected services or resources.



Leverage Hardware Capabilities

ARM TrustedZone: This feature from ARM separates the processor and someperipherals into two zones: a trusted one and a non trusted one. Secrets can then be stored inside the trusted zone to avoid disclosure. This technology deeply inserted within SoC design makes it efficient to keep hardware secret.

Secure RAM: Any encrypted secrets should be decrypted before usage. Nevertheless, decrypted secrets should be kept invisible on any hardware bus. To achieve this, the SoC provides secured RAM inside the trusted zone.

Crypto co-processor. Measurements of integrity for installed programs imply computations of hashes like: SHA256 or MD5. Certifications of origin for downloaded packages imply signature checks. Secure transmissions (SSL/TLS) or disk encryptions require encryption like AES. All these cryptographic tasks are highly accelerated by using built-in processor instructions or coprocessors.

Connectivity with the Outside World

Network Connections

Any connection to the external world introduces new security risks and potential security flaws:

- **Tapping/Spying**: connections need to be encrypted to avoid spying or identity usurpation (spoofing).
- **Privacy enforcement**: end-users love automatic pairing, but how can we detect intrusions? Device pairing should include identification to prevent cross-user spying.
- **Abnormal behavior**: unfriendly applications should not override expected behavior. Access to network resources should be restricted by application using permissions for both incoming and outgoing streams.
- **Open software doors**: any non-necessary resource should be locked down, especially facilities for debugging, development, and testing. System should not expose unexpected services to their external ends. Programs should not be able to open backdoors. However, there are still cases when allowing programs to create open connection entry points makes sense.
- **Denial of Service**: automotive systems should resist DoS-attack-like flooding.
- **Intrusion detection**: Security may lower end-user experience; for convenience they may choose to open multiple doors, reasoning that “I'm not a target” or “It won't happen to me”. Security intrusions or abnormal behaviours must be reported for the user to understand risks and take appropriate actions.
- **Include a firewall and an updated blacklist**: Firewall and blacklist are common techniques for filtering content for child protection. It should also include an updated list of blacklisted servers.
- **Implement Content Security Policy**: To avoid cross-scripting attack, the W3C defines CSP10, content security policy. With CSP web pages declare Internet resources as dependencies. Applying such a policy would allow detection of applications requesting access to blacklisted origins.

Removable Storages

Removable storage is formatted using FAT file systems because it ensures having a wide range of readers. Unfortunately, FAT was designed far before the generalization of DAC/MAC access control mechanisms, which make it difficult or even impossible to secure such devices.

Removable devices always create some extra security threats. If an SD card is used as a disk extension, what happens when it is removed? Should it be encrypted to protect private data? Could it create a security flaw within any other IVI systems? In a perfect world, any removable device would be encrypted. But in reality, until there is a generalization of crypto-accelerators and transparently encrypted file systems, removable devices will remain a threat to integrity and their usage should be limited.

Connectivity to the Cloud

HOW DO WE PROTECT USER PRIVACY?

HOW MUCH SHOULD THE CLOUD BE TRUSTED?

Upgrading Software

Upgrading Baseline System

Upgrading Applications

Very Long Time Support

Thank you!